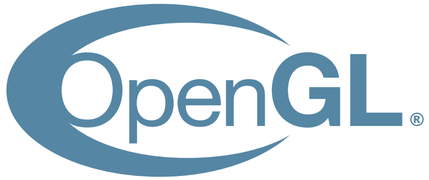
Crat3r
#01 - Introduction to Vulkan and GPU Debugging

Erik Eismann, Marc Stamminger

October 13, 2026

Graphics API Overview



- Cross-platform
- Easy for beginners
- High CPU overhead
- Limited control
- State not obvious



- Cross-platform
- Low overhead
- Explicit control
- Hard to learn
- Very verbose



- Modern features
- DX 11: high level
- DX 12: low level
- Windows/Xbox only
- Proprietary



- Apple only
- Proprietary

OpenGL

High-level API. The driver does a lot of the work for you.

- Uses a global state machine bound to one thread.
- Synchronization is handled implicitly.
- Memory management and placement is done by the driver.
- GLSL code is directly handed to the driver and compiled at runtime.

OpenGL Example

```
unsigned int framebuffer;
glGenFramebuffers(1, &framebuffer);
glBindFramebuffer(GL_FRAMEBUFFER, framebuffer);

unsigned int textureColorbuffer;
glGenTextures(1, &textureColorbuffer);
glBindTexture(GL_TEXTURE_2D, textureColorbuffer);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, SCR_WIDTH, SCR_HEIGHT, 0, GL_RGB, GL_UNSIGNED_BYTE, NULL);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, textureColorbuffer, 0);

unsigned int rbo;
glGenRenderbuffers(1, &rbo);
glBindRenderbuffer(GL_RENDERBUFFER, rbo);
glRenderbufferStorage(GL_RENDERBUFFER, GL_DEPTH24_STENCIL8, SCR_WIDTH, SCR_HEIGHT);
glFramebufferRenderbuffer(GL_FRAMEBUFFER, GL_DEPTH_STENCIL_ATTACHMENT, GL_RENDERBUFFER, rbo);

glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

OpenGL Example

```
1 #define GL_TEXTURE_MAG_FILTER     0x2800
2 #define GL_TEXTURE_MIN_FILTER     0x2801
3
4 #define GL_SRC_ALPHA              0x0302
5 #define GL_ONE_MINUS_SRC_ALPHA    0x0303
```

OpenGL Example

```
glBindFramebuffer(GL_FRAMEBUFFER, framebuffer);
glEnable(GL_DEPTH_TEST); // enable depth testing (is disabled for rendering screen-space quad)

glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

glUseProgram(ID);

glUniformMatrix4fv(glGetUniformLocation(ID, "view"), 1, GL_FALSE, &view[0][0]);
glUniformMatrix4fv(glGetUniformLocation(ID, "proj"), 1, GL_FALSE, &proj[0][0]);

glBindVertexArray(cubeVAO);
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, cubeTexture);
glUniformMatrix4fv(glGetUniformLocation(ID, "model"), 1, GL_FALSE, &model[0][0]);
glDrawArrays(GL_TRIANGLES, 0, 36);

glBindVertexArray(0);

glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

Vulkan

Low-level API. Everything the OpenGL driver did implicitly is now your job.

- The app holds opaque handles to everything.
- Work is recorded into command buffers and submitted to queues.
- Synchronization is explicit.
- Memory is allocated manually.
- Shaders are compiled to SPIR-V ahead of time and baked into pipelines.

$\Rightarrow$ less CPU overhead and multithreaded operations

However: *Initially, your Vulkan code will not be faster than the equivalent OpenGL.*

Vulkan API Call Format

```
typedef struct VkImageCreateInfo {
    VkStructureType          sType;
    const void*              pNext;
    VkImageCreateFlags       flags;
    VkImageType              imageType;
    VkFormat                 format;
    VkExtent3D               extent;
    uint32_t                 mipLevels;
    uint32_t                 arrayLayers;
    VkSampleCountFlagBits    samples;
    VkImageTiling            tiling;
    VkImageUsageFlags        usage;
    VkSharingMode            sharingMode;
    uint32_t                 queueFamilyIndexCount;
    const uint32_t*          pQueueFamilyIndices;
    VkImageLayout            initialLayout;
} VkImageCreateInfo;

VkResult vkCreateImage(
    VkDevice                                device,
    const VkImageCreateInfo*                pCreateInfo,
    const VkAllocationCallbacks*            pAllocator,
    VkImage*                                pImage
);
```

https://docs.vulkan.org/refpages/latest/refpages/source/vkCreateImage.html

Vulkan API Call Format

```
const VkImageCreateInfo imageInfo = {
    .sType                 = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO,
//  .pNext                 = nullptr,
//  .flags                 = 0,
    .imageType             = VK_IMAGE_TYPE_2D,
    .format                = VK_FORMAT_R8G8B8A8_SRGB,
    .extent                = {1000, 1000, 1},
    .mipLevels             = 1,
    .arrayLayers           = 1,
    .samples               = VK_SAMPLE_COUNT_1_BIT,
    .tiling                = VK_TILING_OPTIMAL,
    .usage                 = VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT,
    .sharingMode           = VK_SHARING_MODE_EXCLUSIVE,
//  .queueFamilyIndexCount = 0,
//  .pQueueFamilyIndices   = nullptr,
    .initialLayout         = VK_IMAGE_LAYOUT_UNDEFINED,
};

if (vkCreateImage(device, &imageInfo, nullptr, &image) != VK_SUCCESS) {
    ...
}
```

Use braced initialization to zero out structs!

pNext and sType

```
1 typedef struct VkBaseInStructure {
2     VkStructureType                 sType;
3     const struct VkBaseInStructure* pNext;
4 } VkBaseInStructure;
5
6 typedef struct VkBaseOutStructure {
7     VkStructureType                 sType;
8     struct VkBaseOutStructure*      pNext;
9 } VkBaseOutStructure;
```

- First two arguments of most Vulkan structures.
- `sType` is used by the driver to identify structures when passed as opaque pointers. Always has to be set. Pattern: `VkImageCreateInfo` → `VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO`.
- `pNext` is used when additional configuration is needed, for example for certain extensions. Also used to create a variable length linked list (e.g. list of enabled extensions).

https://docs.vulkan.org/guide/latest/pnext_and_stype.html

Correct API usage practices

- Initialize all variables and structs.
- Use the correct sType for each structure.
- Verify correct pNext chain usage, nulling it out when not needed.
- There are no default values in Vulkan.
- Use correct enum, VkFlag, and bitmask values.
- (Consider using a type-safe Vulkan wrapper, eg. Vulkan.hpp for C++)
- Check function return values, eg VkResult.
- Call cleanup functions where appropriate.

https://docs.vulkan.org/guide/latest/common_pitfalls.html

Default Values

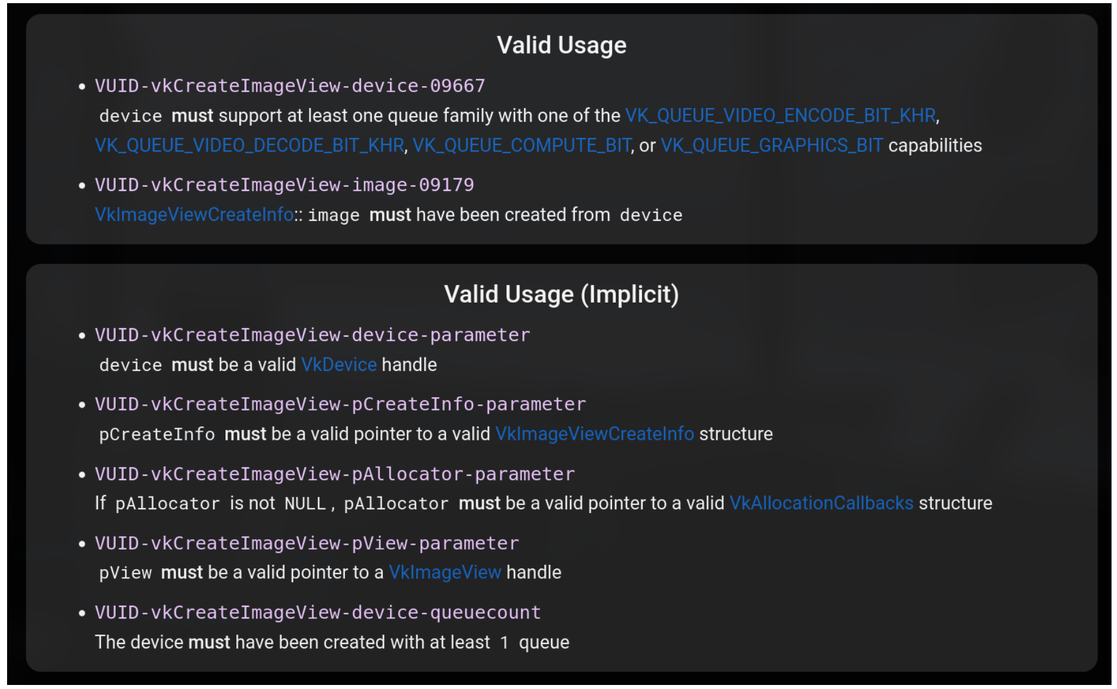
```
typedef enum VkImageTiling {
    VK_IMAGE_TILING_OPTIMAL = 0,
    VK_IMAGE_TILING_LINEAR = 1,
    VK_IMAGE_TILING_DRM_FORMAT_MODIFIER_EXT = 1000158000,
    VK_IMAGE_TILING_MAX_ENUM = 0x7FFFFFFF
} VkImageTiling;
```

```
typedef enum VkSampleCountFlagBits {
    VK_SAMPLE_COUNT_1_BIT = 0x00000001,
    VK_SAMPLE_COUNT_2_BIT = 0x00000002,
    VK_SAMPLE_COUNT_4_BIT = 0x00000004,
    VK_SAMPLE_COUNT_8_BIT = 0x00000008,
    VK_SAMPLE_COUNT_16_BIT = 0x00000010,
    VK_SAMPLE_COUNT_32_BIT = 0x00000020,
    VK_SAMPLE_COUNT_64_BIT = 0x00000040,
    VK_SAMPLE_COUNT_FLAG_BITS_MAX_ENUM = 0x7FFFFFFF
} VkSampleCountFlagBits;
```

Remember: *There are no default values in Vulkan.*

Valid Usage

- Valid Usage is a *‘set of conditions that must be met in order to achieve well-defined run-time behavior in an application’*. The driver *‘doesn’t waste time checking for valid input’*.




```
1 VkResult vkCreateImageView(
2     VkDevice                                    device,
3     const VkImageViewCreateInfo*                pCreateInfo,
4     const VkAllocationCallbacks*                pAllocator,
5     VkImageView*                                pView
6 );
```

- Valid Usage lists are extremely detailed for more complex endpoints.
- Breaking any rule leads to undefined behavior.

https://docs.vulkan.org/guide/latest/validation_overview.html
https://docs.vulkan.org/refpages/latest/refpages/source/vkCreateImageView.html

Vulkan Validation Layers

- Additional layer between API call and driver that check for valid usage.
- Track creation and destruction of objects to find resource leaks.
- Taxing on performance. Should not be enabled in release software.
- **Very important** when developing Vulkan.

```
1 Validation ERROR: [ VUID-vkCmdCopyBufferToImage-dstImage-00177 ] | MessageID = 0xe95493e7
2 vkCmdCopyBufferToImage(): dstImage (VkImage 0x3a000000003a) was created with VK_IMAGE_USAGE_SAMPLED_BIT
3 but requires VK_IMAGE_USAGE_TRANSFER_DST_BIT. The Vulkan spec states: dstImage must have been created
4 with the VK_IMAGE_USAGE_TRANSFER_DST_BIT usage flag set (https://docs.vulkan.org/spec/latest/chapters/
5 copies.html#VUID-vkCmdCopyBufferToImage-dstImage-00177)
```

Automatically enabled in our exercise framework, if installed with the Vulkan SDK.
Important: Do not continue implementing until all validation errors have been fixed.

https://github.com/KhronosGroup/Vulkan-ValidationLayers

Graphics Debugging

Graphics Debugging

- Debugging GPU graphics application is difficult.
  - No step-by-step debugger for shaders
  - Print debugging even less useful
  - Small mistakes can stop any visual output

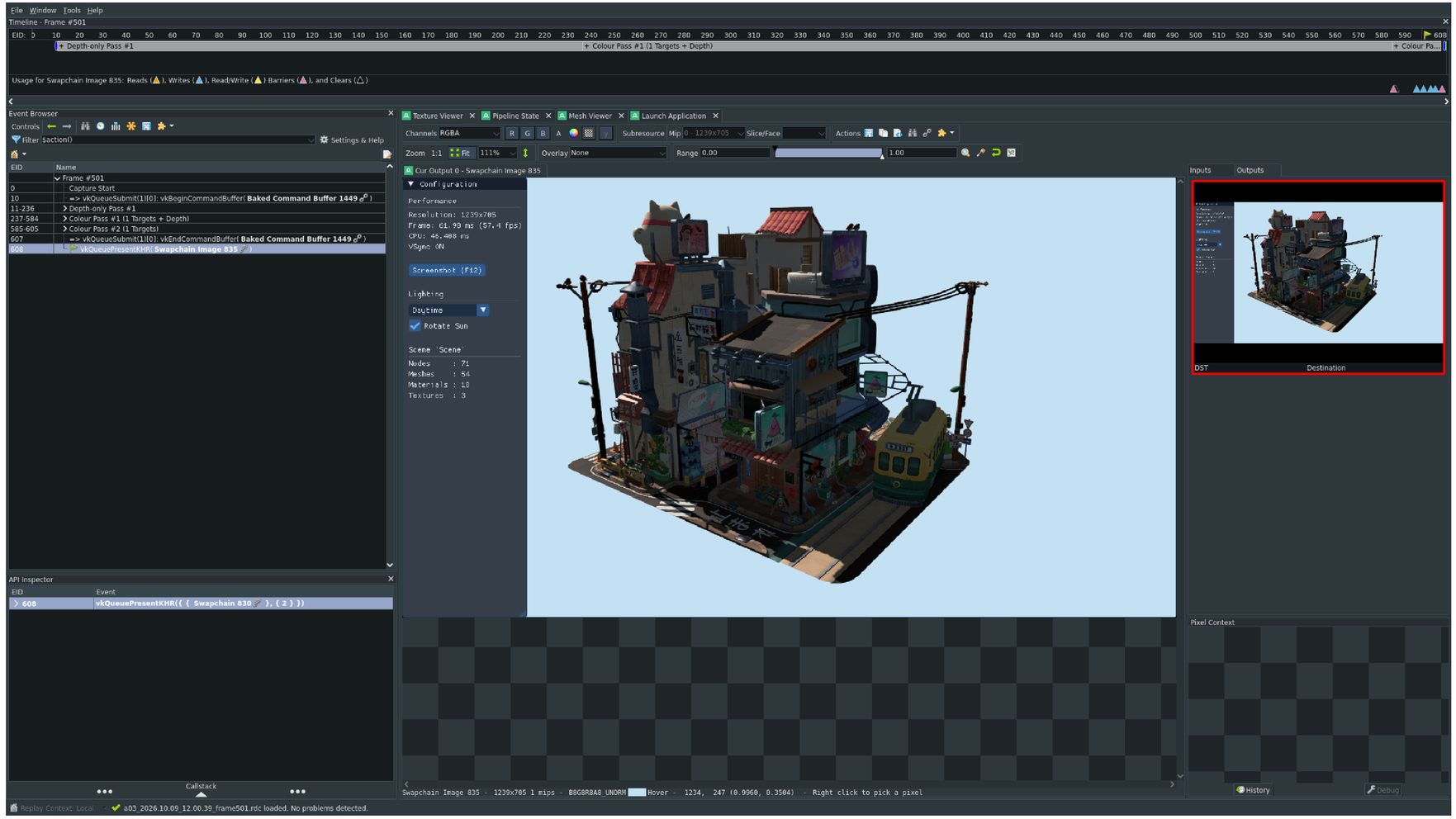
- Use a graphics debugging tool
  - Records API calls and their arguments
  - Shows the contents of buffers and images
  - Details exactly how the frame is rendered step-by-step
  - Some support performance profiling
  - Much more

Graphics Debugging Tools

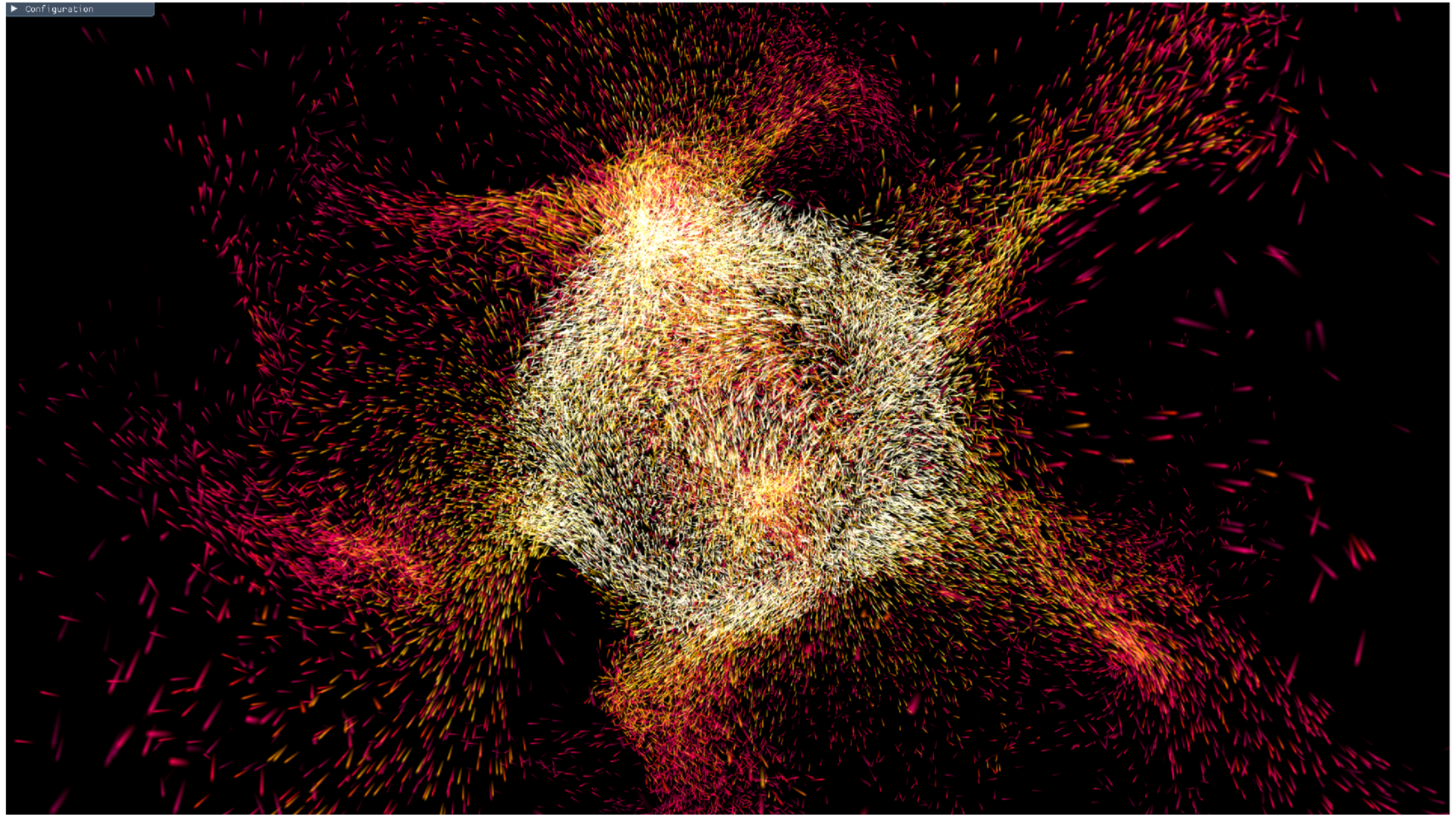
- **RenderDoc**
  - free and open source
  - supports all major graphic card vendors, APIs, and operating systems
  - excellent single frame debugging
  - relatively easy to use

- **NVIDIA Nsight Graphics**
  - proprietary
  - NVIDIA cards only, supports Windows and Linux
  - more confusing user interface than RenderDoc
  - integrates more tightly with NVIDIA cards
  - supports RTX and CUDA

RenderDoc Demo



First Excercise



Next Time

- Compute Shaders, Descriptors and Images